

VDR-LLM-Prolog: Functional Remainder Hardware

Adaptive Precision Through Structural Information in Silicon

Registry: [\[HOWL-VDR-23-2026\]](#)

Series Path: [\[HOWL-VDR-1-2026\]](#) → [\[HOWL-VDR-2-2026\]](#) → [\[HOWL-MATH-3-2026\]](#) → [\[HOWL-MATH-4-2026\]](#) → ... → [\[HOWL-VDR-14-2026\]](#) → ... → [\[HOWL-VDR-21-2026\]](#) → [\[HOWL-VDR-22-2026\]](#) → [\[HOWL-VDR-23-2026\]](#)

DOI: 10.5281/zenodo.20252877

Date: May 2026

Domain: Computer Architecture / Adaptive Precision Arithmetic

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Opus 4.6.

Abstract

The VDR arithmetic system represents every value as a triple [Value, Denominator, Remainder] where the Remainder slot can hold a callable function that produces exact rational values at any requested depth. This functional remainder mechanism — specified in [\[HOWL-VDR-1-2026\]](#) and used throughout the system for transcendental evaluation — has implications for hardware that prior papers in the series did not explore. On the VDR-22 integer-native ASIC [\[HOWL-VDR-22-2026\]](#), each Q335 Integer Unit already contains a 384-bit ALU with 1-2 cycle multiply and free power-of-two division via fixed wiring. This paper specifies a Functional Remainder Unit (FRU) that extends each QIU to evaluate functional remainders — Taylor recurrences, Newton iterations, and series summations — in hardware using the existing ALU, without round-tripping to the host processor. The FRU adds approximately 500,000 transistors per QIU (3.4% die area increase across 5,120 units) and enables three capabilities that the base VDR-22 chip cannot provide: hardware-native exact exponential softmax at competitive latency with float implementations (25-40 nanoseconds for 1,024 logits versus host-bound milliseconds without the FRU), continuous per-step remainder resolution during training that replaces periodic Q-basis reprojection stalls with microsecond-level maintenance, and complete Prolog unification over active VDR values carrying nonzero remainders. At single-query scale, the FRU does not change wall-clock latency — the language model forward pass at approximately 30 microseconds per command token dominates primitive execution at 1-100 nanoseconds by $300\text{-}30,000\times$. At datacenter scale with millions of concurrent sessions, the FRU eliminates host round-trips for remainder resolution, keeping the entire rule-driven execution path on the data-plane chip and removing the serialization bottleneck that would otherwise limit throughput as accumulated Prolog rules handle an increasing fraction of work autonomously. The paper specifies the FRU microarchitecture, traces the full inference chain with adaptive precision, analyzes the datacenter

throughput implications, and identifies the boundary between what the FRU changes (capability and throughput at scale) and what it does not (single-query latency).

1. The Remainder Slot as Precision Control Surface

The VDR triple $[V, D, R]$ has been the foundation of this system since its first specification. V and D are integers forming a rational fraction. R carries exact unresolved structure — the portion of a computation’s result that the current denominator frame could not absorb. R is the only slot that nests, the only slot that recurses, and the only slot that carries active computational state.

When R is zero, the triple is closed — it behaves as the rational number V/D . Arithmetic on closed triples is standard rational arithmetic, and the Q335 fixed frame ($D = 2^{335}$) makes this efficient: addition is one integer add, multiplication is one integer multiply followed by bit extraction at position 335, and the bit extraction is free in hardware because it is fixed wiring.

When R is nonzero, the triple is active — it carries information beyond what the Q335 frame absorbed. R can take three forms: an atomic integer, a composite structure (integer base plus child VDR triples), or a functional remainder — a callable function that produces an exact VDR triple at any requested depth.

The functional remainder is where the VDR system differs most fundamentally from every other arithmetic system. A float value has no concept of “what precision I could achieve if asked.” It has a fixed mantissa width and rounding error that is unknown and unrecoverable. An arbitrary-precision rational has exact precision but no mechanism for deferred computation — the value is either computed or it is not. A VDR functional remainder carries a recipe for computing exact rational values to arbitrary depth, where each depth produces a complete exact value (not an approximation converging to a limit), and the choice of depth is a runtime decision.

Square root of 2 via Newton iteration: each depth doubles the number of correct digits. Depth 8 gives over 100 digits. Exponential via Taylor series: each additional term refines the exact rational partial sum. Sine, cosine, logarithm — all expressible as functional remainders producing exact rationals at any requested depth. The convergence rates vary (Newton is quadratic, Taylor for exp is super-geometric, Taylor for ln is geometric) but the mechanism is uniform: call the function with a depth parameter, receive an exact rational.

On the VDR-22 ASIC without functional remainder hardware, when a computation produces an active value with a functional remainder in the R slot, the QIU stores the remainder metadata in its 4KB SRAM and either carries it forward as unevaluated metadata or signals the host processor to resolve it in software. For the Q335 workloads specified in VDR-22 — batch arithmetic, attention scores, fact matching — this rarely matters. The Q335 frame provides 100 digits of precision, and typical computations produce closed results. The expected active spill rate is below 1%.

The question this paper addresses is what happens when the hardware can evaluate functional remainders natively, without host involvement. Not because the base chip lacks adequate precision — Q335’s 100 digits exceed any practical need. But because the functional remainder is a structural information channel that the hardware can act on, and acting on it enables capabilities the base chip cannot provide.

2. What the FRU Is

2.1 Architecture

The Functional Remainder Unit is a small sequencer attached to each QIU that drives the QIU’s existing ALU through a recurrence loop. It does not contain its own multiplier or adder. It reuses the QIU’s 384-bit carry-select adder (1 cycle) and 384-bit parallel multiplier (1-2 cycles). What it adds is the control logic to execute iterative computations — Taylor term recurrences, Newton iterations, and convergent series — as hardware-managed loops rather than host-orchestrated instruction sequences.

The FRU contains four components.

A recurrence register file holds four 384-bit values: the current term in the series or iteration, the accumulated sum or iterate, the input value x , and a divisor or scaling factor. These are dedicated registers separate from the QIU’s general register file, avoiding contention when the FRU operates concurrently with the QIU’s normal instruction stream.

A small-integer reciprocal table stores the reciprocals of integers 1 through 64 as Q335 values. This table occupies 64×48 bytes = 3,072 bytes in dedicated SRAM. Its purpose is to replace Q335 division-by- k (which would require a full divmod operation) with Q335 multiplication-by-reciprocal (1-2 cycles using the existing multiplier). For Taylor series, each term involves division by the term index k : $\text{term}_k = \text{term}_{\{k-1\}} \times x / k$. The reciprocal table turns this into two multiplies: $\text{term}_{\{k-1\}} \times x$, then $\times \text{reciprocal}[k]$.

A convergence comparator is a single 384-bit magnitude comparison (identical to the QIU’s existing WCMP instruction) that checks whether the current term is below a configurable precision threshold stored in a dedicated threshold register. This determines when the FRU stops iterating — when the next term’s contribution would not change the result at the required precision level.

A loop controller is an 8-bit counter, a function tag decoder (reading the function type from the R slot’s flags field), and a depth limit register. The function tag identifies which recurrence to execute (exp, sqrt, log, sin, cos, or generic series). The depth limit caps the maximum number of iterations regardless of convergence. The total loop controller logic is approximately 2,000 transistors.

2.2 Resource Cost

Component	Transistors	Area at 4nm (mm ²)
Recurrence registers (4 × 384-bit)	36,864	0.0003
Reciprocal table (3 KB SRAM)	147,456	0.0012
Convergence comparator	50,000	0.0004
Loop controller + tag decoder	2,000	<0.0001
Interconnect to QIU ALU	~260,000	0.0021
Per-FRU total	~496,320	~0.004
5,120 FRUs (full chip)	~2.54B	~20.5

The FRU increases the per-QIU transistor count from approximately 7.1M to approximately 7.6M — a 7% increase per unit. At the full-chip level, the increase is from 68B to 70.5B transistors, and from 581 mm² to 601 mm² die area. This is a 3.4% area increase.

2.3 New Instruction

The FRU adds one instruction to the VDR-22 ISA:

Opcode	Mnemonic	Format	Cycles	Description
0x35	FEVAL	R	Variable	Evaluate functional remainder in R slot to depth specified in Id register, using function tag from R flags

FEVAL reads the function tag from the active remainder’s flags field, loads the input value x from the remainder’s V field into the FRU’s recurrence registers, and executes the corresponding recurrence loop using the QIU’s ALU. When the loop terminates (convergence threshold met or depth limit reached), the FRU writes the accumulated result back to the destination V register and the residual remainder (if any) back to the destination R register.

The instruction is variable-latency because different functions require different numbers of iterations and each function’s iteration count depends on the input value and depth parameter. The QIU pipeline stalls during FEVAL execution, the same way it stalls during WMUL. No other instruction issues until the FEVAL completes.

2.4 Recurrence Patterns

The FRU supports six function tags, each corresponding to a specific recurrence:

Tag	Function	Recurrence	Cycles per Term	Depth for 100 Digits
0x01	exp(x)	term k = term {k-1} $\times x \times$ reciprocal[k]	3-4 (2 muls + 1 add)	~45
0x02	sqrt(a)	$x_{n+1} = (x_n + a \times$ reciprocal of(x_n)) $\times$ reciprocal[2]	~4 (1 mul + 1 reciprocal mul + 1 add + 1 shift)	~8
0x03	ln(1+x)	term k = -term {k-1} $\times x$; sum += term k $\times$ reciprocal[k]	3-4 (1 mul + 1 reciprocal mul + 1 add)	~340 at x=1; ~33 with argument reduction
0x04	sin(x)	term k = -term {k-1} $\times x^2 \times$ re- ciprocal[2k] $\times$ recipro- cal[2k+1]	5-6 (3 muls + 1 add + sign)	~45
0x05	cos(x)	term k = -term {k-1} $\times x^2 \times$ reciprocal[2k- 1] $\times$ recipro- cal[2k]	5-6 (3 muls + 1 add + sign)	~45
0x06	generic	term k = f(term {k-1}, x, k) via microcode in instruction BRAM	Variable	Variable

The generic tag (0x06) reads a short microcode sequence from the QIU's instruction BRAM that defines the recurrence step. This enables Borwein acceleration for zeta, hypergeometric series for elliptic integrals, and any other convergent computation expressible as a fixed-step recurrence. The microcode is loaded by the batch dispatcher alongside the main microprogram.

3. Adaptive Precision in the Inference Chain

The functional remainder gives every value in the system a precision dial. The FRU gives the hardware the ability to turn that dial. This section traces what this means through the transformer inference path — not as a fixed pipeline, but as operations the language model may select through command tokens, executed by primitives on the QIU array.

3.1 The Common Path: $R = 0$

For the vast majority of Q335 operations in a transformer forward pass, the Remainder is zero. Multiplication of two Q335 values produces a 768-bit product; SHR335 extracts the quotient (bits above 335) and remainder (bits below 335). When the remainder bits are all zero — which happens whenever the product’s lower 335 bits are zero — the result is closed. No nesting, no FRU involvement, no overhead.

The expected closed rate depends on the values being multiplied. For Q335 constant arithmetic ($\pi + e$, $3 \times \zeta(3)$), the rate is effectively 100% for addition and varies for multiplication. For trained model parameters, which start as small rationals and accumulate through gradient updates, the closed rate is above 99% at typical denominator growth levels.

On the common path, the VDR-22 + FRU chip behaves identically to the base VDR-22. The FRU adds zero overhead for closed values. The convergence comparator checks the CLOSED flag in 1 cycle as part of the normal pipeline — the same check the base chip already performs via the JCLOSED instruction.

3.2 Attention Score Computation

The language model may emit command tokens requesting $Q \times K^T$ computation for attention. This is a matrix of dot products, each involving d_k multiplications and additions.

Each Q335 multiplication in the dot product produces a quotient and a remainder. The QIU checks whether the remainder is zero (1 cycle). If zero, the QIU accumulates the quotient into the running sum and continues. If nonzero, the QIU has a decision point — and this is where adaptive precision operates.

For attention scores that will feed into softmax, the absolute precision of each score matters less than the relative ordering. A remainder of magnitude $2^{(-100)}$ on a score of magnitude $2^{(10)}$ is irrelevant to the ranking. The convergence comparator checks the remainder magnitude against a context-dependent threshold. For attention score intermediates, the threshold is set to the minimum score difference that would change the softmax ranking — a value the host configures before the computation based on the model’s characteristics.

When the remainder is below threshold, the QIU discards it and continues with the closed quotient. No SRAM allocation, no nesting overhead. When the remainder exceeds threshold — rare for well-conditioned attention — the QIU stores it in remainder SRAM and the FEVAL instruction is available if downstream operations need the additional precision.

The result: attention scores computed at full Q335 throughput (1-2 cycle multiply, free divmod) with per-value precision certificates. Each score carries either a zero remainder (exact within the Q335 frame) or a stored remainder with known magnitude (exact within a quantified bound). No float system provides this information.

3.3 Softmax: Surrogate and Exact Exponential

The VDR system specifies five softmax implementations (VDR-14 Appendix E). The two most relevant for hardware are the rational surrogate (SM2: quadratic kernel, zero transcendentals) and the truncated Taylor exponential (SM1: exact rational partial sums).

Rational surrogate on VDR-22 + FRU: unchanged from the base chip. Each output $= (z_i - \max + c)^2 / \sum (z_j - \max + c)^2$. Two multiplies (square and normalization) plus a reduction for the denominator sum. Approximately 20 cycles per row of 1,024 logits across 5,120 QIUs. At 2 GHz: approximately 10 nanoseconds. The FRU is not involved. This remains the lowest-latency path and produces exact sum-to-one with zero transcendentals.

Exact exponential softmax on VDR-22 + FRU: this is the new capability. Each logit passes through the FRU's exp recurrence. The FRU evaluates Taylor terms until the convergence comparator determines sufficient precision for the softmax context.

The adaptive depth per element is the key property. After subtracting the row maximum, shifted logits range from 0 to large negative values. The FRU handles each case differently:

Shifted logit of 0: $\exp(0) = 1$ exactly. The FRU recognizes the zero input and returns 1 in 1 cycle. No iteration.

Shifted logit of -1: the Taylor series converges quickly. The FRU evaluates approximately 6 terms at 3-4 cycles each: approximately 20 cycles.

Shifted logit of -5: approximately 12 terms: approximately 45 cycles.

Shifted logit of -10: approximately 18 terms: approximately 70 cycles.

Shifted logit of -20: this value contributes negligibly to the softmax sum. The FRU evaluates 4-5 terms, confirms the contribution is below the ranking threshold, and stops. Approximately 16 cycles. The remainder carries a functional remainder recording exactly what precision was achieved.

Float hardware computes the same polynomial approximation for every logit regardless of its magnitude. The FRU does less work for values that need less precision and more work for values that need more, directed by the structural information in the remainder.

After all exp evaluations complete, the QIU array performs a global reduction (7 cycles for the exact sum across all QIUs), computes the reciprocal of the sum via FRU Newton iteration (8 iterations $\times$ 4 cycles = 32 cycles, computed once and broadcast), and multiplies each exp value by the reciprocal (1-2 cycles per element).

Total latency for exact exp-softmax over 1,024 logits: the critical path is the longest single exp evaluation (approximately 70 cycles for the most negative shifted logit) plus the reduction (7 cycles) plus the reciprocal (32 cycles) plus normalization (2 cycles). Approximately 111 cycles at 2 GHz: approximately 56 nanoseconds. This is competitive with float exp-softmax on current GPUs (100-200 nanoseconds on H100 including warp divergence from the special function unit), and the result sums to exactly 1/1.

The language model does not choose between surrogate and exact exp-softmax at the instruction level. Both are primitives in the 448-builtin vocabulary (B116 vdr_softmax_surrogate and B115 vdr_softmax). The LLM emits a command token for whichever the orchestration logic selects. The QIU array executes whichever it receives. The FRU enables the exact exp path to execute natively rather than falling back to software.

3.4 Feedforward and Activation

Linear transforms (matrix multiply) follow the same pattern as attention: Q335 multiplication with per-value remainder checking, FRU available for resolution when needed, threshold filtering for intermediates.

ReLU activation is the simplest operation in the chain: compare the value to zero, output the value if positive, output zero if negative. This is one cycle on the QIU comparator. The FRU is relevant only in the edge case where $V = 0$ and the sign depends on the remainder. The QIU checks: $V > 0$? Output V with R . $V < 0$? Output zero. $V = 0$? The FEVAL instruction resolves one level of the remainder tree to determine the sign. Expected frequency of the $V = 0$ case: below 0.01% of values. The FRU adds exact ReLU behavior at the decision boundary at negligible average cost.

Rational scaling (the VDR replacement for layer normalization) requires computing the mean absolute value and dividing each element by it. The mean absolute value is a reduction (exact sum via the reduction network, 7 cycles) followed by division by the element count (exact, since the count is an integer). The reciprocal of the mean absolute value is computed by the FRU via Newton iteration: 8 iterations at 4 cycles each = 32 cycles, computed once and broadcast to all elements. Each element is then multiplied by the reciprocal: 1-2 cycles. Total: approximately 40 cycles for the normalization, less than 20 nanoseconds. No square root. No approximation. Exact.

3.5 What Adaptive Precision Is Not

Adaptive precision via the FRU is not mixed-precision training as practiced in float systems. Float mixed-precision selects a precision level (FP16, FP32, BF16) per tensor or per operation, statically, based on empirically determined heuristics about which layers tolerate lower precision. The selection is coarse-grained (entire tensors), predetermined (before training begins), and uninformed (the system does not know how much precision was lost at any point).

FRU adaptive precision operates per value, per operation, dynamically, directed by exact structural information that each value carries. The granularity is individual scalar values.

The timing is runtime, after each operation produces its result. The information source is the remainder itself — a precise description of what the Q335 frame did not absorb. The decision (resolve further or accept current precision) is a 1-cycle hardware comparison of known quantities, not a heuristic.

The FRU does not make VDR approximate. Every value at every point is exact: V/D plus the structural information in R. The FRU determines whether to evaluate the structural information further based on what the next operation requires. The value does not become more or less exact — it is always exact. The question is whether the evaluated precision is sufficient for the current computational context.

4. Training with Continuous Remainder Resolution

4.1 The Remainder Accumulation Problem

During training, each parameter undergoes repeated multiply-accumulate operations: gradient computation (chain rule through the computation graph) and parameter updates (SGD: $W \leftarrow W - lr \times grad$). Each multiplication potentially produces a nonzero remainder. Over hundreds of training steps, remainders can accumulate — each step’s remainder nesting on top of the previous step’s.

On the base VDR-22 without FRUs, remainder accumulation is managed by periodic Q-basis reproject: when a parameter’s denominator exceeds the declared budget, the system rounds the value to the nearest Q335 representable value, records the exact error bound, and resets the remainder to zero. This is the mechanism specified in VDR-14 (B136 `vdr_reproject_qbasis`). Reprojection is correct — the error bound is exact and recorded — but it is a full-tensor operation that must process every parameter. For a 10-million parameter model, reprojection costs milliseconds when it triggers.

VDR-4 observed that denominators grow from approximately 2^{10} at initialization to approximately 2^{45} after thousands of steps. The Q335 frame at 2^{335} provides 290 orders of magnitude of headroom. Reprojection triggers are infrequent. But when they do trigger, the stall is noticeable — a millisecond-scale pause in an otherwise microsecond-per-step training loop.

4.2 FRU Continuous Resolution

With FRUs, the remainder from each multiplication can be resolved immediately — in the same pipeline cycle sequence as the multiplication itself. After WMUL produces a 768-bit result and SHR335 extracts quotient and remainder, the QIU checks whether the remainder is nonzero. If nonzero, FEVAL resolves it: the FRU evaluates the remainder value against the Q335 frame and either determines it is negligible (below the precision threshold, discard) or folds it back into the quotient as an exact correction.

The folding operation is a Q335 addition of the resolved remainder contribution to the quotient: 1 additional cycle. The total cost of a Q335 multiply with continuous remainder resolution is: 1-2 cycles (multiply) + 0 cycles (SHR335) + 1 cycle (check) + 0-2 cycles

(resolve if nonzero) = 2-5 cycles. At the expected spill rate below 1%, the average cost is approximately 2.02 cycles — negligibly more than the base multiply.

The result: remainder nesting never accumulates. Each multiply’s remainder is resolved inline. The parameter denominators stay within the Q335 frame indefinitely. Q-basis reprojection never triggers because there is nothing to reproject. Training throughput is smooth — no periodic stalls, no full-tensor sweeps, no checkpoint-dependent performance variation.

For 10 million parameters across 5,120 QIUs at approximately 2 cycles per multiply: $10M \times 2 / 5,120 =$ approximately 3,906 cycles per training step for the forward pass multiplications. At 2 GHz: approximately 2 microseconds. With continuous remainder resolution adding approximately 0.02 cycles per multiply on average: approximately 39 additional cycles total, approximately 20 nanoseconds. The remainder resolution cost is below measurement noise.

4.3 Exact Gradient Information at Truncation Boundaries

When the FRU truncates a functional remainder evaluation at a chosen depth (because the convergence comparator determined sufficient precision), the residual remainder is itself a functional remainder — it carries the recipe for computing additional precision on demand. For inference, this additional precision is never needed. For training, it contains exact information about the gradient at the truncation boundary.

The gradient of a truncated Taylor series with respect to its input is the truncated derivative series — also a functional remainder, also exact at every depth. The FRU’s truncation does not introduce gradient error in the way that float rounding introduces gradient error. The truncation boundary is a precise, inspectable, reproducible computational decision, and the gradient through that boundary is exactly computable. This is a property no float system can provide: exact gradients through precision boundaries.

Whether this property has practical consequences for training dynamics at scale is an open empirical question — one that can only be investigated with exact arithmetic, which is the point.

5. Prolog Unification Over Active Values

5.1 The Completeness Problem

Prolog unification of two VDR fractions tests equality by cross-multiplication: $a/b = c/d$ if and only if $a \times d = c \times b$. On the base VDR-22, the CROSS_MUL instruction handles this in 2-3 cycles for closed values ($R = 0$ on both sides). For active values ($R \neq 0$ on either side), the cross-multiplication of the V/D components may produce equal products even when the full values (including remainders) differ, or unequal products when the full values are actually equal after remainder resolution.

Without FRUs, active-value unification falls back to the host: the host resolves both remainders in software, producing closed values, then the QIU performs CROSS_MUL on the closed results. This is correct but requires a host round-trip per unification involving active values.

5.2 FRU-Enabled Active Unification

With FRUs, the QIU can resolve active values inline. When CROSS_MUL detects that one or both operands are active (checking the CLOSED flag), it triggers FEVAL on each active operand to resolve the remainder to the current precision threshold. The resolved values are then cross-multiplied. If the cross-products are equal, the fractions are equal to the resolved precision. If unequal, the fractions are definitively unequal (the difference exceeds the resolved remainder).

The edge case — cross-products equal but remainders might distinguish the values at greater depth — is handled by the convergence comparator. If the remainder magnitudes after resolution are below the threshold where their contribution could change the cross-product comparison, the equality is definitive. If not, the FRU resolves one additional depth level and retries. In practice, Q335 precision (100 digits) makes this edge case vanishingly rare — two values would need to agree to 100 digits and differ only in the 101st for the edge case to activate.

The result: Prolog unification is complete over the entire VDR value space, not just the closed subspace. Constraint evaluation (does the sum of these probabilities equal exactly 1?), fact matching (does this metric value match the threshold?), and rule firing (does this condition hold?) all work correctly on active values without host involvement.

5.3 Impact on the Accumulation Curve

The VDR-19 SRE accumulation curve shows that by investigation 50, 115 Prolog rules auto-fire and 88% of routine work is automated. Each rule firing involves fact matching and unification — comparing stored facts against rule head patterns. If any fact or pattern contains an active VDR value (a measurement with a remainder from sensor conversion, a computed statistic with a remainder from aggregation), the unification must handle active values.

Without FRUs, each active-value unification during rule evaluation requires a host round-trip. At investigation 50 with 115 rules firing, this could mean dozens of host round-trips per investigation — individually fast (microseconds each) but collectively creating control-plane contention at scale.

With FRUs, every rule evaluation completes entirely on the QIU array. The rules fire, facts are matched, active values are resolved inline, and the evaluation result is available without crossing the host-device boundary. The rule-driven execution path stays entirely in the data plane.

6. Single-Query Performance Analysis

At single-query scale, the FRU does not change wall-clock latency for any VDR-LLM-Prolog workload. This section explains why, with specific numbers.

6.1 The Bottleneck Ratio

The language model forward pass generates approximately 8 tokens per command token. For a 1-billion parameter model at Q335 on the VDR-22 chip, with approximately 50 tokens of active context (state is in knowledge bases, not the context window), each forward pass takes approximately 30 microseconds.

Primitive execution takes 1-100 nanoseconds depending on the operation: fact query across 200 facts at 10 nanoseconds, Q335 multiply at 1 nanosecond, softmax over 100 logits at 10 nanoseconds, dot product for H=64 at 32 nanoseconds.

The ratio: each command token costs 30,000 nanoseconds of LLM forward pass and 1-100 nanoseconds of primitive execution. The primitives are $300\text{-}30,000 \times$ faster than the LLM's decision to invoke them.

6.2 FRU Impact on Single-Query

Without FRU, a primitive involving remainder resolution round-trips to the host: add approximately 1-10 microseconds per host round-trip. This is still $3\text{-}30 \times$ faster than the LLM forward pass. The host round-trip is invisible at single-query scale.

With FRU, the same operation completes in 10-100 nanoseconds on the QIU. This is faster than the host round-trip by $10\text{-}100 \times$. But since both are faster than the LLM forward pass, the improvement is invisible to the user. The investigation that takes 23 milliseconds of LLM time with the base chip takes 23 milliseconds with the FRU chip.

6.3 Where Single-Query FRU Value Exists

The FRU's value at single-query scale is not latency. It is capability.

Without FRUs, some computations fall back to software: exact exp-softmax, exact logarithm for cross-entropy loss, exact square root for distance computations, and remainder resolution for active-value Prolog unification. These software fallbacks work correctly but require the host CPU to participate in the data path. The system is functionally complete but architecturally impure — the data plane cannot execute all primitives independently.

With FRUs, every primitive in the 448-builtin vocabulary executes natively on the QIU array. The host CPU manages orchestration (the control plane) and never touches the data path. This is architectural completeness: the silicon can execute everything the specification defines.

Whether this architectural completeness matters at single-query scale is debatable. What it enables at scale is not.

7. Datacenter-Scale Throughput Analysis

At datacenter scale with millions of concurrent sessions, the FRU's elimination of host round-trips changes the throughput ceiling.

7.1 The Serialization Problem Without FRU

The VDR-19 accumulation curve shows that mature systems handle 93% of work through automated Prolog rule evaluation without LLM involvement. Each rule evaluation is a chain of primitive calls — fact matching, unification, arithmetic operations, constraint checks — executing on the QIU array.

Without FRUs, when a rule-driven primitive chain encounters a functional remainder that needs resolution (a transcendental evaluation, an active-value comparison, a remainder that affects the computation outcome), the chain pauses. The QIU signals the host. The host resolves the remainder in software. The host returns the result. The chain resumes.

At single-query scale, this round-trip is invisible — microseconds against a 30-millisecond investigation. At millions of concurrent sessions, each generating hundreds of rule-driven primitive chains per second, the host CPU becomes a serialization point. Millions of round-trips per second compete for host CPU cycles, host memory bandwidth, and host-device communication bandwidth.

7.2 The Data-Plane Solution With FRU

With FRUs, the rule-driven execution path never leaves the QIU array. The FEVAL instruction resolves any functional remainder inline. Prolog unification over active values completes on-chip. Transcendental evaluations for constraint checking complete on-chip. The entire rule evaluation — from fact match through unification through arithmetic through constraint check through result storage — executes as a continuous stream of QIU instructions with no host involvement.

The host CPU handles only LLM forward passes (for the 7% of work requiring LLM judgment), KB tree management (creating and scoping knowledge bases), session orchestration (snapshots, clones), and command token parsing. The data plane is self-sufficient for all numeric computation including transcendentals and remainder resolution.

7.3 Throughput Projections

One VDR-22 + FRU chip at 2 GHz with 5,120 QIUs:

Workload	Compute Limit	Memory Limit (4.9 TB/s)	Effective Throughput
Rule evaluations (100 ops each)	51B/sec	24.5B/sec (at 200 B/fact)	~24B/sec (memory-bound)

Workload	Compute Limit	Memory Limit (4.9 TB/s)	Effective Throughput
SRE investigations (1,500 ops each)	3.4B/sec	16.3B/sec	~3.4B/sec (compute-bound)
Concurrent sessions with 93% automation	—	—	See below

For concurrent sessions with 93% rule automation: each session generates approximately 55 LLM tokens (7% of 769) requiring LLM forward passes, plus approximately 500 rule-driven primitive operations (the 93% handled by accumulated rules). The LLM tokens are the bottleneck per session — 55 forward passes at 30 microseconds each = 1.65 milliseconds of LLM time per investigation. The 500 rule-driven operations at approximately 2 nanoseconds average = 1 microsecond total primitive time. The LLM dominates by $1,650 \times$.

But the LLM forward passes and the rule-driven primitives execute on different hardware. The LLM cluster generates command tokens. The QIU chip executes rule-driven chains independently. At 1,000 concurrent sessions, the QIU chip is at approximately 0.001% utilization for rule-driven work. At 1,000,000 concurrent sessions, the QIU chip is at approximately 1% utilization.

The QIU chip reaches meaningful utilization only when the LLM cluster is large enough to generate millions of concurrent investigation streams. At that scale, one QIU chip serves the rule-driven primitive work for approximately 3 million concurrent investigations per second. The limiting factor is HBM bandwidth for KB fact access, not compute for arithmetic or remainder resolution.

7.4 LLM Cluster Reduction

The accumulation curve’s deepest economic impact at datacenter scale is reducing the LLM cluster size. Without rule accumulation, every command in every investigation requires an LLM forward pass. With 93% rule automation, only 7% of commands require LLM involvement.

For a datacenter serving 10 million investigations per hour: without accumulation, the LLM cluster processes approximately 2.14 billion command tokens per hour (769 per investigation $\times$ 2.78M/hour). With 93% accumulation, the LLM cluster processes approximately 150 million command tokens per hour. The LLM cluster is $14 \times$ smaller for the same throughput.

The QIU chip handles the rule-driven 93% at negligible compute cost relative to its capacity. The FRU ensures this 93% executes entirely on-chip without host serialization.

The economic result: the expensive resource (LLM compute) shrinks by $14 \times$, the inexpensive resource (QIU compute) absorbs the difference, and the system gets cheaper as rules accumulate further.

8. Exp-Softmax vs Surrogate: An Engineering Choice, Not a Forced Compromise

Previous papers in this series presented the rational surrogate softmax as the primary path and the truncated Taylor exponential as an alternative with tradeoffs. With FRU hardware, the tradeoff profile changes enough that both are viable first-class options.

8.1 Comparison on VDR-22 + FRU

Property	Surrogate (SM2)	Exact Exp (SM1 + FRU)
Transcendentals	None	Taylor exp via FRU
Sum-to-one	Exact (1/1)	Exact (1/1)
Monotonicity	Yes	Yes
Equal logits $\rightarrow$ equal outputs	Yes (1/n each)	Yes (1/n each)
Gradient shape	Quadratic landscape	Exponential landscape
Latency (1,024 logits, VDR-22 + FRU)	~10 ns	~56 ns
FRU required	No	Yes
Adaptive depth	N/A	Yes — less work for well-separated logits
Per-value precision information	No (closed computation)	Yes (R slot carries truncation metadata)
Empirical training validation	Validated at toy scale (VDR-4)	Not validated at scale

8.2 When Each Is Appropriate

The surrogate is faster, simpler, and validated. For inference where latency matters and the softmax is not the focus of investigation, the surrogate is the correct choice. It requires no FRU hardware and produces exact results.

The exact exponential is appropriate when the gradient landscape matters — specifically, when investigating whether training instabilities are artifacts of approximate softmax gradients. VDR-4 identified this as future work item FW4: “with exact gradients and updates, determine if training instabilities are arithmetic artifacts or algorithmic.” The exact expo-

nential with FRU hardware makes this investigation feasible at model scales large enough to exhibit the instabilities in question.

The language model does not need to understand this distinction. Both are primitives in the vocabulary. The orchestration logic (human-configured or rule-driven) selects the appropriate softmax variant based on the context — training investigation versus inference deployment.

9. Updated Chip Specifications

Property	VDR-22 Base	VDR-22 + FRU	Change
Transistors	68B	70.5B	+3.7%
Die area	581 mm ²	601 mm ²	+3.4%
TDP	400 W	410 W	+2.5%
Q335 closed arithmetic throughput	5.1T muls/sec	5.1T muls/sec	Unchanged
Q335 add throughput	10.2T adds/sec	10.2T adds/sec	Unchanged
SHR335 (free divmod)	0 gates, 0 power	0 gates, 0 power	Unchanged
exp(x) depth 10	Host-bound ($\sim \mu s$)	$\sim 640M$ evals/sec (~ 25 cycles)	Native
exp(x) depth 45 (100 digits)	Host-bound ($\sim ms$)	$\sim 36M$ evals/sec (~ 180 cycles)	Native
sqr(x) 100 digits	Host-bound	$\sim 1.6B$ evals/sec (~ 32 cycles)	Native
ln(x) with argument reduction	Host-bound	$\sim 480M$ evals/sec (~ 130 cycles)	Native
sin(x)/cos(x) depth 45	Host-bound	$\sim 28M$ evals/sec (~ 225 cycles)	Native
Softmax surrogate (1,024 logits)	~ 10 ns	~ 10 ns	Unchanged
Softmax exact exp (1,024 logits)	Host-bound	~ 56 ns	Native
Prolog unification (active values)	Host-bound	$\sim 4-8$ cycles on-chip	Native
Remainder resolution (per value)	Host-bound	$\sim 2-5$ cycles on-chip	Native
Training reprojection stalls	Periodic ($\sim ms$)	Eliminated (continuous $\sim ns$)	Continuous

10. Limitations and Honest Boundaries

The FRU does not change the single-query latency of any VDR-LLM-Prolog investigation. The LLM forward pass dominates primitive execution by $300\text{--}30,000\times$ and the FRU makes the already-negligible primitive time marginally more negligible. Users of single-query systems will observe no performance difference between VDR-22 and VDR-22 + FRU.

The FRU’s adaptive depth is controlled by a precision threshold that must be configured correctly for each computational context. A threshold set too aggressively (too large) causes the FRU to discard remainders that affect downstream results. A threshold set too conservatively (too small) causes the FRU to evaluate more terms than necessary. The system specification provides default thresholds per operation type (softmax, training update, constraint evaluation), but domain-specific workloads may require tuning. This is analogous to learning rate tuning in conventional training — a configuration decision, not a fundamental limitation.

The FRU’s reciprocal table covers integers 1 through 64. Taylor series for functions like $\log(1+x)$ at x near 1 converge slowly — approximately 340 terms for 100 digits. The reciprocal table does not cover term indices above 64, meaning terms 65-340 require a different approach (computing the reciprocal via Newton iteration in the FRU, adding approximately 32 cycles per term). With argument reduction (reducing x to a small value before applying the series), the number of terms drops to approximately 33, well within the table’s range. Argument reduction is not performed by the FRU — it is a host-orchestrated preprocessing step. For applications requiring log evaluation of arbitrary arguments, the host must preprocess.

The generic function tag (0x06) reads microcode from the QIU’s instruction BRAM. The instruction BRAM capacity is 4KB (512 instructions at 32 bits each). Complex recurrences requiring more than approximately 50 instructions per step may exceed the available microcode space when combined with the main microprogram. This limits the complexity of custom functional remainder evaluations to relatively simple recurrences. Borwein acceleration for zeta (the most complex functional remainder in the VDR specification) fits within this budget. More complex functions may require multi-pass evaluation orchestrated by the host.

The FRU adds 3.4% die area and approximately 2.5% TDP. These are small but nonzero costs paid by every QIU regardless of whether it executes FEVAL instructions. For workloads that never encounter functional remainders (pure closed Q335 arithmetic with no transcendental evaluation), the FRU is unused silicon. The cost is mitigated by the small absolute area (0.004 mm^2 per QIU) and the fact that the FRU’s recurrence registers provide additional temporary storage that the QIU can use for non-FRU purposes if desired.

The FRU does not make VDR-LLM-Prolog practical for production-scale language model training. The memory constraint identified in VDR-22 (96 GB HBM3 limiting model size to tens of millions of parameters for exact training) is unchanged. The FRU

improves training smoothness (eliminating reprojection stalls) but does not change the fundamental memory-capacity limitation.

11. The Architectural Thesis in Silicon

The VDR-1 paper stated: “The remainder is not error. It is not residue. It is not what’s left over after rounding. It is a first-class operational rule that tells the system what to do next with what remains unresolved in the current denominator frame.”

Every subsequent paper in the series applied this principle to a different domain: knowledge bases, Prolog, grammar systems, safety, alignment, performance, deployment, self-extension. The principle remained the same: the remainder carries structural information, and systems that can read and act on structural information outperform systems that discard it.

The FRU is this principle expressed in transistors. The remainder SRAM stores the structural information. The convergence comparator reads it. The recurrence register file acts on it. The loop controller determines when sufficient action has been taken. The QIU’s existing ALU does the actual computation. The result is a value that is always exact, with precision determined dynamically by what the computation requires, directed by information the value itself carries.

Float hardware rounds and discards. The information about what was lost is unrecoverable. VDR hardware rounds and remembers. The FRU can recover the information when the computation needs it. This is not a difference of degree — faster rounding or more bits of precision. It is a difference of kind — a value that knows what it does not yet know, and hardware that can compute the answer on demand.

The common path ($R = 0$, closed, exact within Q335) runs at full throughput with zero FRU overhead. The uncommon path ($R \neq 0$, structural information available) runs at near-full throughput with the FRU resolving what’s needed in 2-32 cycles depending on the function and depth. The hardware never does unnecessary work because the remainder tells it what is necessary. The hardware always does sufficient work because the remainder tells it what is sufficient.

This is what adaptive precision means in an exact arithmetic system. Not “choose FP16 or FP32.” Not “quantize to INT8.” Not “mixed precision with loss-scaling heuristics.” Per-value, per-operation, hardware-determined precision, directed by exact structural information, producing exact results with known and recoverable residuals. The FRU is a 3.4% area investment that makes this operational.

Links

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Denominator, Remainder. Exact Finite Arithmetic in Irreducible Triple Form. DOI: 10.5281/zenodo.15302702

[[HOWL-VDR-4-2026](#)] Exact-Fraction Language Model Architecture. From Arithmetic Library to Working Transformer in 24 Modules.

[[HOWL-VDR-13-2026](#)] VDR in Physical Computation. Exact Arithmetic Where It Matters.

[[HOWL-VDR-14-2026](#)] VDR-LLM-Prolog. A Complete System Specification for Exact Arithmetic Language Models with Structural Provenance. DOI: 10.5281/zenodo.20232194

[[HOWL-VDR-18-2026](#)] VDR-LLM-Prolog: Performance. Integer Arithmetic on GPU Hardware.

[[HOWL-VDR-21-2026](#)] VDR-LLM-Prolog on FPGA. Exact Integer Arithmetic in Custom Silicon: A 10-Core Q335 Processor on Zynq-7020.

[[HOWL-VDR-22-2026](#)] VDR-LLM-Prolog on Dedicated Silicon. From FPGA Proof-of-Concept to Integer-Native GPU Architecture.

Appendix A: FRU Recurrence Detail per Function

A.1 Exponential Taylor Recurrence

Step	Register State	Operation	Cycles	Result
Init	term=1, sum=1, x=input, k=1	Load initial values	2	term ₀ =1, sum ₀ =1
k=1	term=1	term × x	1-2	x
k=1	term=x	term × reciprocal[1]	1-2	x (identity)
k=1	sum=1	sum + term	1	1 + x
k=2	term=x	term × x	1-2	x ²
k=2	term=x ²	term × reciprocal[2]	1-2	x ² /2
k=2	sum=1+x	sum + term	1	1 + x + x ² /2
k=n	term {n-1}	term × x × reciprocal[n]	3-4	x ⁿ /n!
k=n	sum {n-1}	sum + term	1	Σ x ^k /k! for k=0..n
Check	term n		term n	< threshold?

Total cycles per term: 4-5 (2 multiplies + 1 add + 1 check). For depth 10: ~45 cycles. For depth 45: ~200 cycles. Reciprocal lookup replaces division — reciprocal[k] is pre-computed as Q335 integer in the 3KB table.

A.2 Square Root Newton Recurrence

Step	Register State	Operation	Cycles	Result
Init	x_0 =initial guess, a=input	Load	2	$x_0 = a$ (or better guess)
n=1	x_0	$a \times \text{reciprocal of}(x_0)$	2	a/x_0 via Newton reciprocal
n=1	a/x_0	$x_0 + a/x_0$	1	$x_0 + a/x_0$
n=1	sum	sum $\times$ reciprocal[2]	1-2	$(x_0 + a/x_0)/2 = x_1$
Check	x_1		$x_1^2 - a$	< threshold?

Total cycles per iteration: ~7 (1 reciprocal mul + 1 add + 1 halving mul + 1 square + 1 sub + 1 compare). Digits double per step. 8 iterations for 100 digits: ~56 cycles. Note: reciprocal_of(x_0) uses a dedicated Newton reciprocal subroutine (4 iterations, ~16 cycles) on the first call; subsequent calls use the prior reciprocal as starting guess (1-2 iterations, ~8 cycles).

A.3 Logarithm Series with Argument Reduction

Phase	Operation	Cycles	Notes
Reduction	Find integer n such that input = $2^n \times (1+x)$ where	x	< 0.5
Reduction	Load ln(2) from shared BRAM	2	LDSHARED instruction
Reduction	$n \times \ln(2)$ as Q335 multiply	1-2	Base contribution
Series init	term=x, sum=x, k=1	2	log1p series: $\sum (-1)^{(k+1)} \times x^k/k$
Series k=n	term $\times (-x)$	1-2	Next power with alternating sign
Series k=n	term $\times \text{reciprocal}[k]$	1-2	Divide by term index
Series k=n	sum + term	1	Accumulate
Series check		term	< threshold?
Final	$n \times \ln(2) + \text{series sum}$	1	Combine reduction with series

Phase	Operation	Cycles	Notes
Total series cycles per term: 4-5. With argument reduction (	x	< 0.5), convergence ratio is 0.5: approximately 33 terms for 100 digits. Total: ~10 (reduction) + ~150 (series) + 1 (combine) = ~161 cycles. Without argument reduction at x near 1: ~340 terms, ~1,530 cycles — functional but slow.	

A.4 Sine via Taylor Odd-Term Recurrence

Step	Register State	Operation	Cycles	Result
Init	term=x, sum=x, $x^2=x \times x$	3	Precompute x^2 for reuse	
k=1 (3rd order)	term=x	term $\times (-x^2)$	1-2	$-x^3$
k=1	term= $-x^3$	term $\times$ reciprocal[2] $\times$ reciprocal[3]	2-4	$-x^3/6$
k=1	sum=x	sum + term	1	$x - x^3/6$
k=n	term {n-1}	term $\times (-x^2) \times$ reciprocal[2n] $\times$ reciprocal[2n+1]	4-6	$(-1)^n$ $x^{(2n+1)}/(2n+1)!$
Check	term n		term n	< threshold?

Total cycles per term: 6-8 (x^2 multiply + 2 reciprocal multiplies + add + check). Sine converges at the same rate as exp (super-geometric). Depth 45 for 100 digits: ~315 cycles. Cosine is identical but uses even-term recurrence starting from term=1.

A.5 Borwein Zeta via Generic Tag

Phase	Operation	Cycles	Notes
Coefficient compute	$d_k = n \times \sum_{i=0..k} (n+i-1)! \times 4^i / ((n-i)! \times (2i)!)$ for $i=0..k$	~20 per k	Via microcode in instruction BRAM
Term compute	$(-1)^k \times (d_k - d_{k-1}) / (k+1)^s$	~8 per k	Power via repeated squaring
Accumulate	sum += term	1	Running sum
Final	-sum / d_n	~3	One division

For n=210 (100-digit convergence): 210 coefficient computations at ~20 cycles + 210 term computations at ~8 cycles = ~5,880 cycles. At 2 GHz: ~2.9 microseconds. Fits within 512-instruction BRAM budget: coefficient computation is ~30 instructions, term computation is ~15 instructions, loop control is ~8 instructions. Total: ~53 instructions.

Appendix B: Reciprocal Table Contents

B.1 Table Layout (3,072 bytes)

Index k	Value (reciprocal of k as Q335)	Numerator Digits	Exact?	Use Case
1	$2^{335} / 1 = 2^{335}$	101	Exact (power of 2 divides 2^{335})	Taylor term k=1 (identity)
2	$2^{335} / 2 = 2^{334}$	101	Exact	Newton halving, Taylor k=2
3	round($2^{335} / 3$)	101	± 1 ULP	Taylor k=3, sine k=1
4	$2^{335} / 4 = 2^{333}$	101	Exact	Taylor k=4
5	round($2^{335} / 5$)	101	± 1 ULP	Taylor k=5
6	round($2^{335} / 6$)	101	± 1 ULP	Sine/cosine k=1 (1/3!)
8	$2^{335} / 8 = 2^{332}$	100	Exact	Taylor k=8

Index k	Value (reciprocal of k as Q335)	Numerator Digits	Exact?	Use Case
16	$2^{335} / 16 = 2^{331}$	100	Exact	Taylor k=16
32	$2^{335} / 32 = 2^{330}$	99	Exact	Taylor k=32
64	$2^{335} / 64 = 2^{329}$	99	Exact	Taylor k=64 (table limit)

B.2 Exactness Analysis

k Range	Power-of-2 k Values (exact)	Odd k Values (± 1 ULP)	Total Exact	Total ± 1 ULP
1-8	1, 2, 4, 8	3, 5, 6, 7	4	4
9-16	16	9-15	1	7
17-32	32	17-31	1	15
33-64	64	33-63	1	31
Total			7	57

For the 57 entries with ± 1 ULP error: the rounding error is at most $2^{(-336)}$, which is approximately $10^{(-101.2)}$. This is 10^{66} times below the Planck length. The error from reciprocal table lookup is negligible in every conceivable physical computation. The VDR system tracks this error through the remainder slot — the reciprocal multiply produces a result in the Q335 frame, and any sub-ULP residual goes to R.

B.3 Memory Layout

Byte Offset	Contents	Size
0x000 - 0x02F	reciprocal[1] (48 bytes, 384-bit Q335)	48 B
0x030 - 0x05F	reciprocal[2]	48 B
0x060 - 0x08F	reciprocal[3]	48 B
...	...	...
0xBD0 - 0xBFF	reciprocal[64]	48 B
Total	64 entries $\times$ 48 bytes	3,072 B

Appendix C: Convergence Threshold Analysis

C.1 Default Thresholds by Operation Context

Context	Threshold (as Q335 numerator magnitude)	Decimal Equivalent	Rationale
Softmax ranking stability	$2^{(335-20)} \approx 2^{315}$	$\sim 10^{(-6)}$ relative	Preserves ranking for logits differing by $>10^{(-6)}$
Attention score intermediate	$2^{(335-30)} \approx 2^{305}$	$\sim 10^{(-9)}$ relative	Conservative; score differences typically $>10^{(-3)}$
Training gradient	$2^{(335-50)} \approx 2^{285}$	$\sim 10^{(-15)}$ relative	Preserves gradient direction for typical learning rates
Constraint evaluation (exact)	0	0 (require R=0)	Sum-to-one and other axiom constraints must be exact
Prolog unification (definitive)	Context-dependent	See C.2	Must be sufficient to resolve equality
Denominator budget check	N/A	N/A	Integer comparison, no threshold
Conservation law verification	0	0	Must be exact equality

C.2 Prolog Unification Threshold Decision

Scenario	Value A	Value B	CROSS MUL Result	Threshold Decision
Both closed	V a/D a	V b/D b	Compare products directly	No threshold — exact
A active, B closed	[V a, D a, R a]	V b/D b	Resolve R a to depth where	R a contribution
Both active	[V a, D a, R a]	[V b, D b, R b]	Resolve both to depth where combined remainder < cross-product difference	Threshold = cross-product difference minus remainder bounds
Equal at Q335 frame	V aD b = V bD a	V aD b = V bD a	Must resolve R a and R b to distinguish	Resolve one additional depth, compare

C.3 Threshold Configuration Mechanism

Parameter	Storage Location	Set By	Modified By
Softmax threshold	FRU threshold register	Host at kernel launch	Per-kernel configuration
Training threshold	FRU threshold register	Host at training step start	Per-step if adaptive
Constraint threshold	Hardwired to 0	Not modifiable	Axiom — always exact
Prolog threshold	Computed per-unification	QIU during CROSS MUL	Automatic from operand values

Appendix D: FRU Power Analysis

D.1 Per-FRU Power Breakdown

Component	Static (μ W)	Dynamic at 2 GHz (μ W)	Notes
Recurrence registers (1,536 bits SRAM)	0.8	12 (when accessed)	6T SRAM cells, leakage + read/write

Component	Static (μ W)	Dynamic at 2 GHz (μ W)	Notes
Reciprocal table (3 KB SRAM)	1.5	8 (per lookup)	Read-only during operation
Convergence comparator	0.3	5 (per comparison)	384-bit magnitude compare
Loop controller	0.1	2 (per cycle when active)	8-bit counter + decoder
Interconnect to QIU ALU	0.5	3 (per data transfer)	Short-range within QIU
Per-FRU total (idle)	3.2 μ W	0	
Per-FRU total (active)	3.2 μ W	~30 μ W	Per active cycle

D.2 System-Wide FRU Power

Scenario	Active FRUs	Duration	FRU Power	System Total (410W)	FRU Fraction
Idle (all FRUs inactive)	0	Continuous	16.4 mW (static only)	400.016 W	0.004%
Softmax exp (100 logits, 10 FRUs)	10	~50 cycles	0.3 mW (burst)	400.0003 W	<0.001%
Batch training (1% spill, ~50 FRUs)	50	~5 cycles/occurrence	1.5 mW (intermittent)	400.002 W	<0.001%
Full exp-softmax (all 5,120 FRUs)	5,120	~100 cycles	153.6 mW (burst)	400.154 W	0.038%

Scenario	Active FRUs	Duration	FRU Power	System Total (410W)	FRU Fraction
Worst case (all FRUs, continuous)	5,120	Continuous	170 mW	400.170 W	0.042%

The FRU's power contribution is negligible in all realistic scenarios. Even the worst case (all 5,120 FRUs active continuously) adds 170 milliwatts to a 400-watt chip — 0.042%. The TDP increase from 400W to 410W in the updated specifications is primarily from the additional static leakage of 2.54 billion transistors, not from dynamic switching during FRU operations.

Appendix E: Adaptive Depth Distribution for Exp-Softmax

E.1 Shifted Logit Distribution (Typical Attention Row, 1,024 positions)

Shifted Logit Range	Expected Count (1,024 positions)	FRU Depth Required	Cycles per Element	Total Cycles (batch)
0 (the maximum)	1	0 (exact 1)	1	1
(-0.5, 0)	~50	3-4 terms	14-18	800
(-2, -0.5]	~200	5-7 terms	22-30	5,200
(-5, -2]	~300	8-12 terms	34-50	12,600
(-10, -5]	~250	13-18 terms	55-75	16,250
(-20, -10]	~150	4-5 terms (threshold cut)	18-22	3,000
< -20	~73	2-3 terms (threshold cut)	10-14	876
Total	1,024			~38,727

Average cycles per element: ~37.8. Across 5,120 QIUs (each handling ~1 element in this row, or multiple rows in batch): critical path is the slowest single element at ~75 cycles = ~37.5 ns.

E.2 Comparison with Uniform-Depth Evaluation

Strategy	Total Cycles (1,024 elements)	Average Cycles per Element	Notes
Adaptive depth (FRU threshold)	~38,727	~37.8	Does less work on negligible values
Uniform depth 10	~46,080	45	Same work for all values
Uniform depth 20	~92,160	90	Same work for all values
Uniform depth 45 (full 100-digit)	~207,360	202.5	Same work for all values
Surrogate (no transcendents)	~20,480	20	Quadratic kernel, no FRU

Adaptive depth saves approximately 16% over uniform depth 10, approximately 58% over uniform depth 20, and approximately 81% over full-precision depth 45. The savings come entirely from the large-negative shifted logits (positions (-20, -10] and below -20) where the threshold cuts evaluation short because those values contribute negligibly to the softmax sum.

E.3 Ranking Stability Verification

Logit Pair Difference	Adaptive Depth Sufficient?	Minimum Depth for Correct Ranking	Notes
> 1.0	Yes at depth 3	3	exp difference detectable at $\sim 10^{(-1)}$ precision
0.1 - 1.0	Yes at depth 5	5	exp difference detectable at $\sim 10^{(-3)}$ precision
0.01 - 0.1	Yes at depth 8	7-8	Need $\sim 10^{(-5)}$ precision
0.001 - 0.01	Yes at depth 10	10	Need $\sim 10^{(-8)}$ precision
< 0.001	Depends on threshold	12+	Near-tied logits; ranking may be meaningless regardless

For practical attention patterns where logit differences below 0.001 are rare, adaptive depth 10 preserves correct ranking for all meaningful distinctions. The threshold mechanism ensures that tied logits (difference below threshold) produce equal softmax outputs — the FRU evaluates both to the same depth and produces identical results, preserving the symmetry property.

Appendix F: Training Remainder Accumulation Simulation

F.1 Remainder Nesting Without FRU (Base VDR-22)

Training Step	Avg Denominator Bits	Max Denominator Bits	Parameters with $R \neq 0$	Nesting Depth (max)	Reprojection Needed?
0 (init)	~10	~12	0%	0	No
100	~15	~20	0.01%	1	No
1,000	~20	~30	0.1%	2	No
5,000	~30	~42	0.5%	3	No
10,000	~35	~48	0.8%	4	Approaching budget
20,000	~40	~55	1.2%	5	Yes — stall
50,000	~45	~65	2.0%	7	Multiple stalls

Reprojection at step 20,000: process all 10M parameters, round to Q335 frame, record error bounds. Cost: $\sim 10M \times 20$ cycles (read + round + write + log) / 5,120 QIUs = $\sim 39,063$ cycles = ~ 20 microseconds. Not catastrophic but a visible stall in an otherwise 2-microsecond-per-step training loop — a $10 \times$ spike.

F.2 Remainder Nesting With FRU (Continuous Resolution)

Training Step	Avg Denominator Bits	Max Denominator Bits	Parameters with $R \neq 0$ (before resolution)	Parameters with $R \neq 0$ (after resolution)	Nesting Depth (max)	Reprojection Needed?
0 (init)	~10	~12	0%	0%	0	No
100	~10	~12	0.01%	0%	0	No
1,000	~10	~13	0.1%	0%	0	No
5,000	~10	~13	0.5%	0%	0	No
10,000	~10	~14	0.8%	0%	0	No

Training Step	Avg Denominator Bits	Max Denominator Bits	Parameters with R $\neq$ 0 (before resolution)	Parameters with R $\neq$ 0 (after resolution)	Nesting Depth (max)	Reprojection Needed?
20,000	~10	~14	1.2%	0%	0	No —
50,000	~11	~15	2.0%	0%	0	never No —
						never

With continuous FRU resolution, the denominator bits stay near their initialization values indefinitely. Each step's multiplication produces a remainder; the FRU evaluates it (2-5 cycles), determines it is below the Q335 precision threshold, and folds it back into the quotient or discards it. The max denominator bits drift upward very slowly (~1 bit per 10,000 steps) from the rare cases where the remainder is large enough to affect the Q335 frame value. Reprojection never triggers.

F.3 Training Step Timing Comparison

Component	Without FRU	With FRU	Difference
Forward multiply (per param)	2 cycles	2 cycles	Same
SHR335 divmod	0 cycles	0 cycles	Same
Remainder check	1 cycle	1 cycle	Same
Remainder resolution (when R $\neq$ 0)	Host round-trip (~5,000 cycles)	FRU inline (~3 cycles)	1,667 $\times$ faster
Average per param (at 1% spill)	$2 + 0 + 1 + 0.01 \times 5000 = 53$ cycles	$2 + 0 + 1 + 0.01 \times 3 = 3.03$ cycles	17.5 $\times$ faster
10M params total	103.5M cycles = 51.8 μ s	30.3M cycles = 15.2 μ s	3.4 $\times$ faster
Periodic reprojection stall	~20 μ s every ~20K steps	Never	Eliminated
Worst-case step time	~72 μ s (step + reprojection)	~15.2 μ s (constant)	4.7 $\times$ faster
Step time variance	High (spikes at reprojection)	Near-zero	Smooth

Appendix G: Prolog Unification Completeness Matrix

G.1 Term-Type Unification Latency With and Without FRU

Term A Type	Term B Type	Without FRU	With FRU	Improvement
Atom	Atom	1 cycle (UATOM)	1 cycle	None
Integer	Integer	1 cycle (WCMP)	1 cycle	None
VDR closed	VDR closed	2-3 cycles (CROSS MUL)	2-3 cycles	None
VDR closed	VDR active	Host (~5,000 cycles)	6-8 cycles (FEVAL + CROSS MUL)	$\sim 700 \times$
VDR active	VDR active	Host (~10,000 cycles)	8-12 cycles ($2 \times$ FEVAL + CROSS MUL)	$\sim 1,000 \times$
KB ref	KB ref	1 cycle (integer compare)	1 cycle	None
List	List	$N \times$ element cost	$N \times$ element cost	Per-element improvement for active elements
Atom	Variable	1 cycle (bind)	1 cycle	None
VDR active	Variable	Host + bind (~5,000 cycles)	FEVAL + bind (~6 cycles)	$\sim 800 \times$

G.2 Impact on Rule Firing Rate

Investigation #	Rules Auto-Firing	Facts with Active Values (est.)	Unification Requiring FRU (est.)	Host Round-Trips Without FRU	On-Chip with FRU
1	0	0%	0	0	0

Investigation #	Rules Auto-Firing	Facts with Active Values (est.)	Unification Requiring FRU (est.)	Host Round-Trips Without FRU	On-Chip with FRU
10	47	~2% of accessed facts	~12	12	0
50	115	~5% of accessed facts	~85	85	0
100	150	~8% of accessed facts	~180	180	0

At investigation 100 with 150 rules auto-firing: 180 host round-trips per investigation at ~5,000 cycles each = 900,000 additional cycles = 450 microseconds of host-mediated overhead. With FRU: 180×10 cycles = 1,800 cycles = 0.9 microseconds. A $500 \times$ reduction in unification overhead for mature rule-accumulated systems.

Appendix H: Instruction BRAM Budget for FRU Microprograms

H.1 Microcode Size per Function Tag

Tag	Function	Instructions in Recurrence Loop	Instructions in Init/Finalize	Total Instructions	BRAM Words (32-bit)
0x01	exp	8 (2 mul + 1 add + 1 check + 4 control)	4 (load x, init term/sum)	12	12
0x02	sqrt	10 (reciprocal + add + halve + square + check + 4 control)	5 (load a, init guess)	15	15
0x03	ln	9 (negate mul + reciprocal mul + add + check + 4 control)	8 (argument reduction)	17	17

Tag	Function	Instructions in Recurrence Loop	Instructions in Init/Finalize	Total Instructions	BRAM Words (32-bit)
0x04	sin	11 (x^2 mul + 2 reciprocal mul + add + sign + check + 4 control)	5 (precompute x^2 , load)	16	16
0x05	cos	11 (same as sin)	5	16	16
0x06	generic	Variable (loaded from main BRAM)	Variable	≤ 50	≤ 50
Total fixed				76	76

H.2 BRAM Capacity Analysis

Usage	Words	Percentage of 4KB BRAM (512 words)
Main microprogram (e.g., prog softmax surr)	~30	5.9%
FRU function tags 0x01-0x05 (fixed)	76	14.8%
FRU generic tag 0x06 (loaded per operation)	≤ 50	$\leq 9.8\%$
Total maximum	~156	30.5%
Remaining for future microprograms	~356	69.5%

The FRU's fixed function microprograms consume 14.8% of the instruction BRAM — well within budget. The remaining 69.5% capacity supports additional main microprograms, more complex generic FRU functions, or expanded main program logic. The BRAM is loaded by the batch dispatcher before each operation, so different operations can use the full capacity independently.

Appendix I: Host Round-Trip Elimination at Scale

I.1 Round-Trip Sources Without FRU

Round-Trip Cause	Frequency per Investigation	Host CPU Cycles per Trip	Total Host Cycles per Investigation
Transcendental evaluation (exp, log, sqrt)	~5	~50,000	~250,000
Active-value Prolog unification	~12 (at investigation 50)	~10,000	~120,000

Round-Trip Cause	Frequency per Investigation	Host CPU Cycles per Trip	Total Host Cycles per Investigation
Remainder resolution for constraint check	~3	~20,000	~60,000
Deep remainder traversal (depth > 4)	~1	~100,000	~100,000
Total per investigation	~21		~530,000

I.2 Concurrent Session Impact Without FRU

Concurrent Sessions	Round-Trips per Second	Host CPU Utilization (at 3 GHz, 8 cores)	Bottleneck?
1,000	21,000	~0.5%	No
10,000	210,000	~4.6%	No
100,000	2,100,000	~46%	Approaching
500,000	10,500,000	~230% (saturated)	Yes
1,000,000	21,000,000	~460% (severely saturated)	Yes

I.3 Concurrent Session Impact With FRU

Concurrent Sessions	Round-Trips per Second	Host CPU Utilization	Bottleneck?
1,000	0	~0% (data plane only)	No
10,000	0	~0%	No
100,000	0	~0%	No
500,000	~500 (deep remainder only)	~0.001%	No
1,000,000	~1,000 (deep remainder only)	~0.002%	No
10,000,000	~10,000	~0.02%	No

With FRUs, the only remaining host round-trip cause is deep remainder traversal beyond depth 4-5, which occurs in approximately 1 in 1,000 investigations. The host CPU is effectively removed from the data path. The serialization bottleneck that saturates at

500,000 concurrent sessions without FRUs does not manifest with FRUs at any tested scale.

Appendix J: Functional Remainder Composition

J.1 Composition Rules in Hardware

Composition	Mathematical Result	FRU Behavior	Cycles
$\exp(a) + \exp(b)$	Not a functional remainder of $(a+b)$	Evaluate both, add results	$2 \times$ FEVAL + WADD
$\exp(a \times b)$	$\exp$ composed with multiply	Multiply first, then FEVAL $\exp$ on result	WMUL + FEVAL
$\text{sqrt}(a \times b)$	sqrt composed with multiply	Multiply first, then FEVAL sqrt on result	WMUL + FEVAL
$\log(\exp(a))$	Identity (a)	FEVAL $\exp$, then FEVAL $\log$; result should equal a	$2 \times$ FEVAL (verify round-trip)
$\exp(\log(a))$	Identity (a)	FEVAL $\log$, then FEVAL $\exp$; result should equal a	$2 \times$ FEVAL (verify round-trip)
$\sin^2(a) + \cos^2(a)$	1 exactly	FEVAL $\sin$, FEVAL $\cos$, WMUL each, WADD; should equal 1	$2 \times$ FEVAL + 2 $\times$ WMUL + WADD

J.2 Composition Depth Management

Scenario	Composed Depth	FRU Strategy	Risk
Single FEVAL on closed value	1	Standard recurrence	None
FEVAL on result of FEVAL	2	Second FEVAL uses first result's precision	Precision compounds — sufficient at Q335

Scenario	Composed Depth	FRU Strategy	Risk
Chain of 3+ FEVALs	3+	Each uses prior result; precision adequate if each step maintains Q335 frame	Precision degrades predictably — R slot tracks
FEVAL on active value from multiply chain	1 + nesting depth	Resolve multiply remainder first, then evaluate function	Total depth = multiply depth + function depth

J.3 Verification Tests for Composition

Test	Input	Expected Output	Verifies
exp(0)	0	1 exactly (R=0)	Base case
exp(ln(2))	ln(2) as Q335	2 $\pm$ Q335 ULP	exp-log round-trip
sqrt(4)	4	2 exactly (R=0)	Perfect square
sqrt(2) ²	fn sqrt(2)	2 $\pm$ Q335 ULP	Square of irrational
sin ² (π /6) + cos ² (π /6)	π /6 as Q335	1 $\pm$ Q335 ULP	Pythagorean identity
exp(a+b) vs exp(a) $\times$ exp(b)	a=1, b=2	Equal $\pm$ Q335 ULP	Exponent law

All tests produce results exact to the Q335 frame (100 digits). The $\pm$ Q335 ULP notation indicates that the result may differ from the mathematical exact value by at most 1 unit in the last Q335 position — this is the fundamental rounding from projection onto the 2^{335} grid, present in all Q335 arithmetic and tracked by the R slot.

Appendix K: Die Photo Partition Estimate (VDR-22 + FRU)

K.1 Area Allocation

Region	Area (mm ²)	Percentage	Contents
SM array (80 SMs)	344	57.2%	5,120 QIUs with FRUs, shared SRAM, instruction cache, schedulers
— QIU ALUs within SMs	164	27.3%	384-bit multipliers, adders, shifters, comparators
— QIU registers within SMs	102	17.0%	V/R register files, FRU recurrence registers
— QIU remainder + FRU SRAM within SMs	56	9.3%	Remainder BRAM, reciprocal tables
— SM shared SRAM	16	2.7%	Q335 constants, predicate lookup, config
— SM control and interconnect	6	1.0%	Schedulers, local reduction, cache
L2 cache	155	25.8%	96 MB SRAM
HBM3 PHY + controllers	24	4.0%	6 HBM3 stack interfaces
Global reduction network	4	0.7%	7-level binary tree of 384-bit adders
PCIe + NVLink	12	2.0%	Host and inter-chip interfaces
Host interface + scheduling	8	1.3%	Global dispatch, interrupt, DMA
Power management	4	0.7%	Voltage regulators, clock distribution
I/O ring + ESD	30	5.0%	Pad ring, electrostatic discharge protection
FRU total (within SM array)	~20.5	3.4%	Recurrence regs + reciprocal table + controller
Total die	601	100 %	

K.2 Transistor Density Comparison

Region	Transistors (B)	Area (mm ²)	Density (M tr/mm ²)	Notes
QIU ALU (compute logic)	13.3	164	81	Lower den- sity — com- plex rout- ing
QIU registers (SRAM)	4.9	102	48	Standard 6T SRAM den- sity
L2 cache (SRAM)	19.2	155	124	High- density SRAM
HBM PHY (mixed)	3.0	24	125	macro Analog + digi- tal mix
FRU (logic + SRAM)	2.5	20.5	124	Mostly SRAM (re- cip- rocal ta- ble)
Full chip average	70.5	601	117	

The FRU's transistor density (124 M/mm²) is higher than the ALU logic (81 M/mm²) because the FRU is predominantly SRAM (reciprocal table at 3KB dominates the transistor count). SRAM packs more densely than random logic at 4nm. This means the FRU's physical area impact is slightly less than its transistor percentage would suggest.

Appendix L: Software Fallback Comparison

L.1 Operations with Software Fallback (Base VDR-22) vs Native (VDR-22 + FRU)

Operation	Software Path (Host CPU)	Latency	Native Path (FRU)	Latency	Speedup
exp(x) depth 10	Zig Taylor loop on ARM	$\sim 2 \mu s$	FEVAL tag 0x01, depth 10	$\sim 20 ns$	$100 \times$
exp(x) depth 45	Zig Taylor loop on ARM	$\sim 10 \mu s$	FEVAL tag 0x01, depth 45	$\sim 90 ns$	$111 \times$
sqrt(x) 100 digits	Zig Newton on ARM	$\sim 5 \mu s$	FEVAL tag 0x02, 8 iters	$\sim 28 ns$	$179 \times$
ln(x) with reduction	Zig series on ARM	$\sim 15 \mu s$	FEVAL tag 0x03, ~ 33 terms	$\sim 80 ns$	$188 \times$
sin(x) depth 45	Zig Taylor on ARM	$\sim 12 \mu s$	FEVAL tag 0x04, 45 terms	$\sim 160 ns$	$75 \times$
cos(x) depth 45	Zig Taylor on ARM	$\sim 12 \mu s$	FEVAL tag 0x05, 45 terms	$\sim 160 ns$	$75 \times$
$\zeta(s)$ via Borwein n=210	Zig Borwein on ARM	$\sim 200 \mu s$	FEVAL tag 0x06, 210 terms	$\sim 2.9 \mu s$	$69 \times$
Active-value unification	Zig resolve + compare	$\sim 5 \mu s$	FEVAL + CROSS MUL	$\sim 8 ns$	$625 \times$

L.2 Conditions Where Software Fallback Remains Necessary

Condition	Why FRU Cannot Handle	Frequency
Remainder depth > 16	FRU remainder SRAM limited to 32 nodes; deep traversal exceeds hardware capacity	$< 0.001\%$ of operations
Reciprocal index > 64	Reciprocal table covers $k=1..64$; higher indices require Newton reciprocal sub-loop	log series without argument reduction
Custom recurrence > 50 instructions	Instruction BRAM budget for generic tag limited to ~ 50 instructions	Exotic series only
Non-convergent series	FRU depth limit prevents infinite loops but cannot detect divergence meaningfully	Programmer error
Function composition requiring intermediate KB access	FRU cannot issue KB queries mid-evaluation	Rare — composition typically on register values

Appendix M: Comparison with Float Special Function Units

M.1 SFU vs FRU Architecture

Property	Float SFU (e.g., NVIDIA SM)	VDR FRU
Purpose	Approximate transcendentals at float precision	Exact rational transcendentals at configurable depth
Method	Polynomial approximation or lookup-interpolation	Taylor/Newton recurrence producing exact partial sums
Precision	Fixed (23-bit mantissa for FP32)	Configurable (depth parameter controls digits)
Error bound	Documented ULP error (1-2 ULP typical)	Exact: R slot carries precise residual
Latency	~20-30 cycles (fixed)	Variable: 1 cycle (exp(0)) to ~225 cycles (sin depth 45)
Throughput per SM	~8 evals/cycle (shared SFU)	64 evals/cycle (1 FRU per QIU)
Power	~10 mW per SFU	~30 μ W per FRU (active)
Die area	~0.1-0.3 mm ² per SFU	~0.004 mm ² per FRU
Deterministic	Yes (same polynomial)	Yes (same recurrence)
Platform-independent	No (vendor-specific polynomial)	Yes (integer arithmetic)
Result type	Float with unknown rounding error	Exact rational with known structural remainder

M.2 Throughput-Precision Tradeoff

Precision Target	Float SFU Evals/sec/SM	FRU Evals/sec/QIU	FRU Advantage
~7 digits (FP32-equivalent)	~500M	~400M (depth 3-4)	Comparable, FRU slightly slower
~15 digits (FP64-equivalent)	~250M (FP64 SFU)	~200M (depth 8)	Comparable, FRU slightly slower
~50 digits	Not possible	~80M (depth 20)	FRU only option
~100 digits	Not possible	~36M (depth 45)	FRU only option
Adaptive (per-value)	Not possible	Variable (1-225 cycles)	FRU only option

At float-equivalent precision levels, the FRU is competitive with (slightly slower than) dedicated SFUs. Beyond float precision, the FRU is the only option. The ability to choose precision per value per operation has no float equivalent.

Appendix N: End-to-End Latency Trace (12-Layer Transformer)

N.1 Per-Layer Breakdown at $d_model=512$, $seq_len=1024$, 8 heads, VDR-22 + FRU

Component	Operation Count	Dominant Cost	QIUs Active	Latency
QKV projection (3 linear)	$3 \times 1024 \times 512$ matmul	512 muls per output element	5,120	$\sim 15 \mu s$
Attention QK^T (8 heads)	$8 \times 1024 \times 1024$ $\times 64$ dot products	64 muls per element	5,120	$\sim 100 \mu s$
Softmax (surrogate)	8×1024 rows $\times$ 1024 logits	20 cycles per element	5,120	$\sim 3 \mu s$
Softmax (exact exp via FRU)	8×1024 rows $\times$ 1024 logits	~ 38 cycles avg per element (adaptive)	5,120	$\sim 6 \mu s$
Value mixing	$8 \times 1024 \times 1024$ $\times 64$	64 muls per output element	5,120	$\sim 100 \mu s$
Output projection	1024×512 matmul	512 muls per element	5,120	$\sim 5 \mu s$
Residual + rational scaling	1024×512 add + reduction + broadcast mul	3 cycles per element + 40 cycles norm	5,120	$\sim 0.1 \mu s$

Component	Operation Count	Dominant Cost	QIUs Active	Latency
Feedforward linear 1 ($4 \times$ expansion)	$1024 \times 512 \times 2048$ matmul	512 muls per element	5,120	$\sim 50 \mu s$
ReLU	1024×2048 compare	1 cycle per element	5,120	$\sim 0.4 \mu s$
Feedforward linear 2	$1024 \times 2048 \times 512$ matmul	2048 muls per element	5,120	$\sim 200 \mu s$
Residual + rational scaling	1024×512 add + norm	Same as above	5,120	$\sim 0.1 \mu s$
Per-layer total (surrogate softmax)				$\sim 474 \mu s$
Per-layer total (exact exp softmax)				$\sim 477 \mu s$

N.2 Full Model Forward Pass

Layers	Per-Layer (surrogate)	Per-Layer (exact exp)	Total (surrogate)	Total (exact exp)	Difference
6	$474 \mu s$	$477 \mu s$	2.84 ms	2.86 ms	+0.7%
12	$474 \mu s$	$477 \mu s$	5.69 ms	5.72 ms	+0.5%
24	$474 \mu s$	$477 \mu s$	11.38 ms	11.45 ms	+0.6%

The difference between surrogate and exact exp-softmax at the full-model level is below 1%. The softmax is a small fraction of total forward pass time, dominated by the matrix multiplications in attention and feedforward. The choice between surrogate and exact exp has negligible impact on inference latency.

N.3 Comparison with Float Reference

Model Config	VDR-22 + FRU (exact, surrogate)	VDR-22 + FRU (exact, exp)	H100 Float16 (approximate)
6-layer, $d=512$, $seq=1024$	2.84 ms	2.86 ms	~ 1.5 ms
12-layer, $d=512$, $seq=1024$	5.69 ms	5.72 ms	~ 3.0 ms

Model Config	VDR-22 + FRU (exact, surrogate)	VDR-22 + FRU (exact, exp)	H100 Float16 (approximate)
24-layer, d=512, seq=1024	11.38 ms	11.45 ms	~6.0 ms
Precision	100 digits exact	100 digits exact	~7 digits (FP16), 23 digits (FP32)
Reproducible	Bit-identical any platform	Bit-identical any platform	Platform-dependent
Provenance per value	Full (R slot + KB)	Full (R slot + KB)	None
Sum-to-one (softmax)	Exactly 1/1	Exactly 1/1	$\sim 1.0 \pm 10^{-7}$

The VDR-22 + FRU forward pass is approximately $1.9 \times$ slower than H100 float16 on a per-token basis. At 85-97% token reduction, the VDR system performs $5\text{-}20 \times$ fewer forward passes per prompt. The net compute per prompt favors VDR from the first prompt, and the advantage widens with conversation length due to flat per-turn cost.

[[HOWL-VDR-23-2026](#)] VDR-LLM-Prolog: Functional Remainder Hardware. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-23-2026>

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-3-2026>

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-14-2026](#)] VDR-LLM-Prolog. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-14-2026>

[[HOWL-VDR-21-2026](#)] VDR-LLM-Prolog on FPGA. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-21-2026>

[[HOWL-VDR-22-2026](#)] VDR-LLM-Prolog on Dedicated Silicon. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-22-2026>

[[HOWL-VDR-4-2026](#)] Exact-Fraction Language Model Architecture. Github:
<https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-4-2026>

[[HOWL-VDR-13-2026](#)] VDR in Physical Computation. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-13-2026>

[[HOWL-VDR-18-2026](#)] VDR-LLM-Prolog: Performance. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-18-2026>